Project Implementation Checklist: Mughal Miniature & Ragamala Analysis & Authentication

This master checklist outlines every task required to implement the end-to-end pipeline specified in your project requirements, , and .

Architecture Flow Overview

Phase 1: Data Ingestion, Extraction & Metadata Parsing

- **1.1 Download and Stage Dataset**
  - Pull dataset archive from Google Drive (`1FA1C4KFiZeWXwwk_Z7ESb4JlkslVKoG6`).
  - Organize local directory structure (`data/raw/images` , `data/raw/captions` , `data/metadata`).
  - Verify image presence across the 3 categories:
    1. Mughal Miniature Paintings
    2. Painting Crops
    3. Ragamala Paintings
- **1.2 Metadata Ingestion & Alignment**
  - Parse `sources.txt` to map provenance, albums, and collections.
  - Parse `metadata.jsonl` and link entries to corresponding `.jpg` files.
  - Parse individual `.txt` caption files; validate structure against `metadata.jsonl` .
  - Generate a unified tabular manifest (`dataset_manifest.parquet` or `csv`) with columns: `image_id` , `file_path` , `category` , `raga` , `school` , `period` , `artist` , `collection_id` , `caption` , `has_box_labels` .
- **1.3 Data Integrity & Validation**
  - Validate image file headers, resolutions, color channels (ensure conversion to RGB if RGBA/CMYK/grayscale).
  - Identify and flag corrupted or unreadable images.

- Map label distributions and identify class imbalances across targets (*Raga*, *School*, *Period*, *Artist*).

Phase 2: Leak-Free Grouped Splitting Strategy

> IMPORTANT
>
> Never use random `train_test_split`. Miniatures from the same manuscript set, series, or album share distinct paletting, paper tone, border decoration, and artist hands. Random splitting causes severe data leakage and artificially inflated metrics.

- **2.1 Identify Grouping Keys**
  - Group by manuscript series, collection, or album ID extracted from `sources.txt` and caption provenance.
- **2.2 Implement Grouped Stratified Partition**
  - Use `StratifiedGroupKFold` or `GroupShuffleSplit` (e.g., 70% Train, 15% Validation, 15% Test).
  - Ensure disjoint groups: paintings from the same manuscript folio never appear in both train and validation/test.
  - Verify label distributions across splits for each task (*raga*, *school*, *period*, *artist*).
  - Export frozen split manifests (`train_split.jsonl`, `val_split.jsonl`, `test_split.jsonl`).

Phase 3: Benchmark Baseline Model (Completed)

- ✓ **3.1 Extract Frozen Vision Embeddings**
  - ✓ Extract representations using frozen CLIP or ViT (`torchvision.models.vit_b_16`, `vit_b_32`, or `timm` models) with full parameter freezing (`requires_grad=False`).
  - ✓ Save embedding cache (`data/cache/embeddings_{split}_{model}.npz`) with automatic hit-validation to prevent redundant feature re-computation.
- ✓ **3.2 Fit Baseline Classifiers**
  - ✓

Train regularized Logistic Regression heads with balanced class weights (`class_weight='balanced'`) on standardized frozen embeddings:

    - Baseline Head 1: Raga classification
    - Baseline Head 2: School classification
    - Baseline Head 3: Period classification
    - Baseline Head 4: Artist attribution

  - ✓ Persist fitted models and label encoders to `models/baseline_classifiers.joblib`.

- ✓ **3.3 Establish Baseline Benchmark Metrics**
  - ✓ Compute Top-1 Accuracy, Macro F1, Weighted F1, and Expected Calibration Error (ECE).
  - ✓ Generate normalized confusion matrix plots (`reports/confusion_matrix_{target}.png`).
  - ✓ Log baseline score tables to `reports/baseline_benchmark.json` and `reports/baseline_benchmark.md`.

Phase 4: Resolution-Preserving Preprocessing & Sliding Windows

- **4.1 Native Resolution Preservation**
  - Retain original image resolution without lossy global resizing.
- **4.2 Image Enhancement Filters**
  - Apply edge-preserving denoising (Non-Local Means / Median filtering).
  - Perform white-balance and color normalization.
  - Apply lighting adjustment (adaptive contrast enhancement, gamma correction).
  - Apply Unsharp Masking to accentuate fine miniature brushwork and contours.
- **4.3 Sliding Window Generation**
  - Configure sliding window parameters:
    - Window size: $512 \times 512$512 × 512 px (adjustable based on average image dimension).
    - Stride: $256$256 px (50% overlap, standard range 30%–70%).
    - Edge padding for border regions.

- Store window coordinate metadata for coordinate mapping back to the full canvas.

Phase 5: Motif & Region Detection (Open-Vocab & Fine-Tuning)

- **5.1 Zero-Shot Open-Vocabulary Detection**
  - Set up Grounding DINO with art-historical and iconographic text prompts:
    - Figures & Personae: *nayaka*, *nayika*, *ascetic*, *krishna*, *attendant*, *dancer*.
    - Flora & Fauna: *peacock*, *deer*, *elephant*, *lotus*, *kadamba tree*, *monsoon clouds*.
    - Material Culture: *veena*, *tambura*, *canopy*, *pavilion*, *lotus pond*, *turban*, *garland*.
- **5.2 Multi-Window Aggregation & Non-Maximum Suppression (NMS)**
  - Project window-level detections back to global canvas coordinates.
  - Apply bounding-box NMS to merge overlapping window proposals.
  - Filter proposals using confidence thresholds.
- **5.3 Ground Truth Comparison & Fine-Tuning Decision**
  - Map detected motifs against available bounding box annotations and cropped images in the dataset.
  - If key iconographic motifs are missed (low recall), fine-tune Grounding DINO / YOLOv8 on box labels.
  - Cache cropped motif patches and bounding box metadata (`type` , `bbox` , `confidence`).

Phase 6: Dual-Branch Feature Engineering & Fusion

Branch 6A: Handcrafted Feature Extraction

- **Color Descriptors (HSV & Lab)**:
  - Hue, Saturation, Value histograms.
  - Dominant color palette extraction (k-means clustering in Lab space).
  - Gold leaf and traditional mineral pigment intensity profiles.
- **Texture Descriptors**:
  - Gray-Level Co-occurrence Matrix (GLCM: contrast, correlation, energy, homogeneity).
  - Local Binary Patterns (LBP) for paper weave and paint stippling.

    - Gabor filter banks at multiple orientations and scales.
- **Brushstroke & Line Morphology**:
    - Canny edge orientation distribution and line curvature.
    - Stroke length, stroke directionality, and contour density.
- **Compositional & Spatial Layout**:
    - Spatial centroids of detected motifs (figures vs. architectural elements).
    - Rule-of-thirds grid alignment, horizon-line ratio, and focal point symmetry.

Branch 6B: Deep Feature Extraction

- **Backbone Feature Generation**:
    - Pass global image and detected motif crops through vision backbone (CLIP ViT / ResNet / EfficientNet).
- **Region Feature Aggregation**:
    - Pool region embeddings using multi-head attention pooling or weighted mean pooling.
- **Global Feature Fusion**:
    - Concatenate L2-normalized deep embeddings with standardized handcrafted feature vectors.
    - Apply dimensionality reduction / linear projection layer if required.

Phase 7: Multi-Head Classifiers & Domain-Aware Training

- **7.1 Multi-Head Architecture**
    - Implement shared fused backbone with 4 independent classification heads:
        1. **Head 1: Raga** (e.g., Bhairava, Megha, Malkaus, Hindol, Dipak, Sri, etc.).
        2. **Head 2: School** (Mughal, Mewar, Bundi, Kota, Kangra, Kishangarh, Malwa, Deccan).
        3. **Head 3: Period** (16th c., early 17th c., late 17th c., 18th c., 19th c.).
        4. **Head 4: Artist Attribution** (Known master artists vs. workshop/school attribution).
- **7.2 Class Imbalance Handling**
    - Compute balanced class weights (`compute_class_weight` / Focal Loss) or apply SMOTE / weighted oversampling on minority ragas and artists.
- **7.3 Domain-Constrained Augmentations**

> **CAUTION**
>
> Color palette is a primary authentic historical signal (e.g., Mewar deep red/yellow vs. Kangra subtle pastel tones vs. Mughal muted realism). Never apply heavy hue shifting, color inversion, or extreme channel mixing.

    - Apply geometric augmentations: random crops, small rotations ($\pm 10^\circ$ ±10∘), subtle perspective transformations.
    - Apply conservative photometric adjustments: small contrast jitter ($\pm 5\%$±5%), minor brightness jitter ($\pm 5\%$±5%).

- **7.4 Training & Validation Execution**
    - Train with Early Stopping, AdamW optimizer, and Cosine Annealing learning rate schedule.
    - Log training dynamics and loss curves per head.

Phase 8: Model Calibration & Uncertainty Estimation

- **8.1 Post-Hoc Temperature Scaling**
    - Optimize temperature parameter $T > 0$T > 0 on the held-out validation set using negative log-likelihood (NLL).
    - Apply learned temperature scaling to test set logits: $\hat{p}_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$p^i = ∑j exp(zj/T)exp(zi/T).
- **8.2 Calibration Error Evaluation**
    - Measure Expected Calibration Error (ECE) across confidence bins.
    - Generate Reliability Diagrams (binned accuracy vs. mean confidence) before and after temperature scaling.
    - Verify that a 70% model confidence score corresponds empirically to ~70% precision.

Phase 9: Multimodal LLM Judge & Contextual Reasoning (Completed)

- ✓
  9.1 Domain Knowledge Base & Retriever Engine
    - ✓
      Built structured knowledge base: Sanskrit Dhyana verses, translations, and mandatory motifs (`knowledge_base/dhyana_verses_ragas.json`).

    - ✓ Built school stylistic signatures, palettes, and master ateliers (`knowledge_base/schools_characteristics.json`).
    - ✓ Built period markers from Akbar 16th c. through 18th c. (`knowledge_base/periods_characteristics.json`).
    - ✓ Built `ArtKnowledgeRetriever` (`knowledge_base/retriever.py`) for semantic and keyword retrieval based on classifier candidates, OCR text, and motifs.
- ✓ **9.2 Multimodal Prompt Engineering & Qwen VLM Reasoning**
    - ✓ Implemented `VLMJudge` and `QwenOllamaVLMClient` supporting local Qwen models with multimodal visual inputs and OCR inscriptions.
    - ✓ Strictly enforced JSON schema, region-specific evidence citations, visible classifier top-3 probabilities, and mandatory `"uncertain"` abstention when evidence is thin.
- ✓ **9.3 Cross-Check & Expert Review Routing**
    - ✓ Built programmatic cross-check: automatically triggers expert review when statistical classifier and VLM disagree or confidence < 0.70.
    - ✓ Implemented `ExpertReviewQueue` generating markdown review cards (`reports/expert_review/review_card_{id}.md`) and summary queues.
- ✓ **9.4 Dual Reasoning Evaluation**
    - ✓ Implemented evaluation harness (`src/test_reasoning.py`): scores both Answer Correctness and Evidence Citation Validity against canonical Sanskrit verses.
    - ✓ Benchmarked and verified on held-out test paintings (`reports/vlm_reasoning_benchmark.md`).

Phase 10: Evaluation, Reporting & Feedback Loop

- **10.1 Quantitative Performance Assessment**
    - Calculate Top-1 Accuracy, Macro F1, and Per-Class F1 across all 4 targets.

    - Plot Confusion Matrices (specifically inspecting confusions between musically and visually related Ragamala families).
    - Verify target accuracy threshold: validate whether the final pipeline achieves the **85% – 90%** target and significantly outperforms the baseline.
- **10.2 Error Analysis & Inspection**
    - Analyze misclassified samples (e.g., distinguishing between Bundi vs. Kotah or related ragas).
    - Inspect motif detector attention maps and attribution gradients.
- **10.3 Human-in-the-Loop (HITL) Feedback Engine**
    - Provide review interface / mechanism for art-historian feedback.
    - Store feedback corrections in structured memory.
    - Establish continuous fine-tuning pipeline for model and prompt refinement.